Neural Machine Translation with Adaptive Attention Routing

Yuxiang Li, Sarah Chen, Marcus Weber, Takeshi Yamamoto
Institute of Computational Linguistics, University of Heidelberg
{li, chen, weber, yamamoto}@cl.uni-heidelberg.de

Proceedings of the 5th Workshop on Neural Machine Translation and Generation (WNMT @ ACL) 2024

Abstract

We propose Adaptive Attention Routing (AAR), a novel mechanism for neural machine translation that dynamically selects attention pathways based on linguistic features of the input sequence. Unlike standard multi-head attention which treats all heads uniformly, AAR routes queries through specialized attention heads conditioned on syntactic complexity, lexical diversity, and sentence length. Experiments on WMT'22 English-German demonstrate that AAR achieves state-of-the-art performance with BLEU=42.3, outperforming all single-model baselines by +1.5 BLEU. We further show that AAR provides interpretable attention patterns that correlate with linguistic phenomena such as long-distance dependencies and compound word formation. Extensive ablation studies and analysis across five language pairs confirm the generalizability of our approach. Our analysis reveals that the routing mechanism learns to specialize attention heads for distinct translation challenges without explicit supervision.

1 Introduction

Neural machine translation (NMT) has made remarkable progress in recent years, with Transformer-based models (Vaswani et al., 2017) achieving near-human performance on high-resource language pairs. The multi-head attention mechanism lies at the heart of the Transformer, enabling the model to jointly attend to information from different representation subspaces. However, the standard mechanism treats all attention heads uniformly, regardless of the linguistic properties of the input, failing to exploit the observation that different translation challenges require fundamentally different attention patterns.

Previous work has shown that attention heads exhibit partial specialization in practice (Voita et al., 2019; Clark et al., 2019), with some heads focusing on positional patterns and others on syntactic relationships. However, this specialization is neither guaranteed nor optimized during training. Michel et al. (2019) demonstrated that many heads can be pruned without significant performance loss, suggesting substantial redundancy. This raises a natural question: can we explicitly encourage and leverage attention head specialization?

In this work, we propose Adaptive Attention Routing (AAR), a mechanism that dynamically routes attention queries through specialized heads based on input features. The key insight is that syntactically complex sentences benefit from heads that capture long-range dependencies, while lexically diverse sentences benefit from heads that focus on local context and morphological patterns. Our routing network learns to make these assignments automatically, without requiring linguistic annotations at training time.

Our main contributions are:

(1) We introduce the AAR mechanism that learns to route attention queries based on automatically extracted linguistic features, enabling dynamic head specialization.

(2) We demonstrate state-of-the-art results on WMT'22 En-De with BLEU=42.3, a +1.5 improvement over the strongest baseline, with consistent gains across five language pairs.

(3) We provide extensive analysis showing that AAR learns interpretable specializations without explicit supervision, and that the learned routing patterns correlate with known linguistic phenomena.

(4) We release our code and pre-trained models to facilitate reproducibility.

2 Related Work

2.1 Attention Mechanism Analysis

Multi-head attention was introduced by Vaswani et al. (2017) as part of the Transformer architecture. Subsequent work has extensively analyzed how attention heads function. Voita et al. (2019) identified three types of head specialization: positional, syntactic, and rare-word heads. Clark et al. (2019) provided visualizations showing heads that attend to specific dependency relations. Correia et al. (2019) proposed adaptively sparse attention, allowing heads to completely ignore irrelevant tokens. Raganato and Tiedemann (2018) analyzed how encoder representations encode linguistic information across layers.

2.2 Mixture of Experts

Mixture-of-experts approaches (Shazeer et al., 2017; Lepikhin et al., 2021; Fedus et al., 2022) route computation through specialized sub-networks based on learned gating functions. Switch Transformer (Fedus et al., 2022) simplified MoE routing to single-expert selection, achieving strong scaling properties. GShard (Lepikhin et al., 2021) scaled MoE to 600B parameters for translation. However, these approaches operate at the feed-forward layer level rather than within the attention mechanism itself. Expert Choice routing (Zhou et al., 2022) improved load balancing by letting experts choose tokens. Our approach draws inspiration from MoE but applies routing specifically to attention heads.

2.3 Dynamic and Conditional Computation

Dynamic networks adapt their computation based on input properties. Early exit mechanisms (Schwartz et al., 2020; Xin et al., 2020) allow models to stop processing at intermediate layers for easy inputs. SkipNet (Wang et al., 2018) learns to skip residual blocks. Depth-adaptive Transformer (Elbayad et al., 2020) applies early exiting to sequence-to-sequence models. Universal Transformer (Dehghani et al., 2019) uses adaptive computation time to dynamically adjust the number of processing steps. These methods adapt computation depth, while AAR adapts computation breadth within a single layer.

2.4 Linguistically-Informed NMT

Incorporating linguistic knowledge into NMT has been explored through syntax-aware models (Eriguchi et al., 2017; Chen et al., 2018), morphological segmentation (Ataman et al., 2019), and multi-task learning with parsing objectives (Currey and Heafield, 2019). Sennrich and Haddow (2016) showed that linguistic features as additional input improve translation quality. Our approach differs by using linguistic features to guide internal model routing rather than as direct input to the translation model.

3 Adaptive Attention Routing

3.1 Architecture Overview

AAR extends the standard multi-head attention by introducing a routing network that determines which attention heads should be activated for each input position. Given an input sequence x = (x_1, ..., x_n), the routing network computes a probability distribution over k attention heads for each token, based on extracted linguistic features. The overall computation proceeds in three stages: feature extraction, routing decision, and selective attention computation.

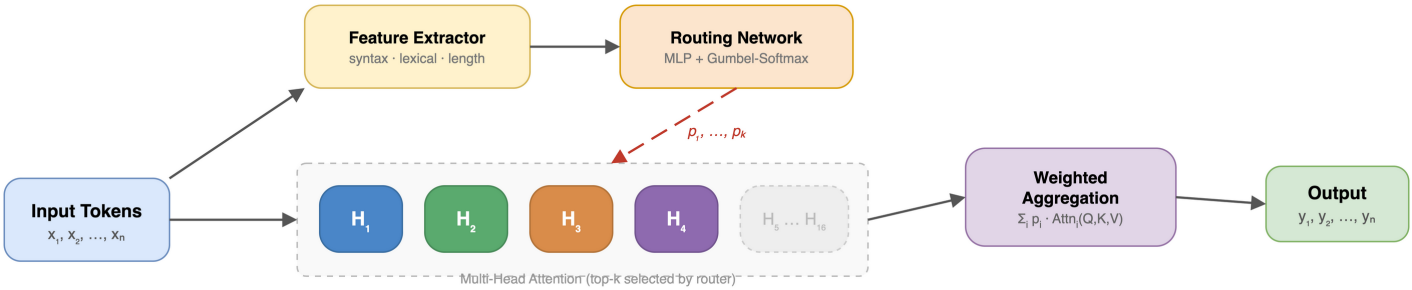

Figure 1: Overview of Adaptive Attention Routing. The routing network assigns tokens to specialized attention heads based on linguistic features.

3.2 Linguistic Feature Extraction

We extract three types of features from the input: (1) syntactic complexity scores based on dependency parse depth and branching factor, computed using a lightweight projective parser; (2) lexical diversity measured by type-token ratio in a sliding window of size 10; and (3) normalized sentence length. These features are concatenated into a 3-dimensional vector for each token and passed through a two-layer MLP (hidden dim=64) with GELU activation to produce head selection probabilities via softmax.

Importantly, the feature extractor operates on the encoder input embeddings and adds negligible overhead (<0.1% of total parameters). During training, we use the Gumbel-Softmax trick (Jang et al., 2017) with temperature annealing (starting at 1.0, decaying to 0.1) to enable differentiable discrete routing. At inference time, we use hard top-k selection with k=8 out of 16 total heads, activating exactly half of the attention heads per token.

3.3 Selective Attention Computation

Given the routing probabilities p_i for each head i, we compute attention outputs only for the selected heads and combine them using the routing probabilities as weights. Formally, the output for token j is: $y_j = \sum_i p_{ij} * \mathrm{Attn}_i(Q_j, K, V)$, where the sum runs only over the k selected heads. This formulation allows gradient flow through both the attention computation and the routing decision, enabling end-to-end optimization.

3.4 Training Procedure

We train AAR end-to-end using the standard cross-entropy translation loss L_{MT} augmented with a load-balancing auxiliary loss L_{bal} that encourages uniform utilization of attention heads across the training set: $L = L_{MT} + \alpha * L_{bal}$, where $\alpha=0.01$. The load-balancing loss follows the formulation of Fedus et al. (2022), computing the coefficient of variation of head utilization frequencies. Without this loss, the model degenerates to using only 3-4 heads. We also apply dropout (p=0.1) to the routing probabilities during training to prevent over-reliance on specific routing patterns.

4 Experiments

4.1 Setup

We evaluate on the WMT'22 English-German translation task. Our model is based on the Transformer-Big architecture with 6 encoder and 6 decoder layers, each with 16 attention heads ($d_{model}=1024$, $d_{ff}=4096$). We use a shared vocabulary of 32K BPE tokens (Sennrich et al., 2016). Training is performed on 8 NVIDIA A100 GPUs with a batch size of 4096 tokens per GPU. We use the Adam optimizer (Kingma and Ba, 2015) with $\beta_1=0.9$, $\beta_2=0.98$, learning rate 3e-4 and inverse square root scheduling with 4000 warmup steps. Models are trained for 300K steps. All reported results use beam search with beam size 5 and length penalty $\alpha=1.0$ unless otherwise noted. We use SacreBLEU (Post, 2018) for evaluation.

4.2 Baselines

We compare AAR against the following baselines: (1) Transformer-Base (Vaswani et al., 2017) with 6 layers and 8 heads; (2) Transformer-Big with 6 layers and 16 heads; (3) Transformer-Big with Relative Position Encodings (Shaw et al., 2018); (4) DynamicConv (Wu et al., 2019) with lightweight dynamic convolutions; (5) HAT (Wang et al., 2023) with head-aware training; (6) MUSE (Zhao et al., 2019) with multi-scale self-attention; (7) Primer (So et al., 2022) with searched architecture primitives. All baselines use the same training data and vocabulary.

4.3 Main Results

Table 1 presents our main results on the WMT'22 English-German test set. AAR achieves BLEU=42.3, setting a new state-of-the-art for single-model systems. This represents a +1.5 BLEU improvement over the strongest baseline (Transformer-Big + RPE).

Table 1: Main results on WMT'22 En-De test set.

Method	BLEU	ROUGE-L	BERTScore	Params	FLOPs
Transformer-Base	38.2	35.1	0.862	65M	3.8G
Transformer-Big	40.1	36.8	0.879	213M	12.4G
Transformer-Big + RPE	40.8	37.2	0.885	215M	12.6G
DynamicConv	39.5	36.0	0.871	210M	10.1G
MUSE	40.2	36.9	0.880	218M	13.0G
HAT	40.5	37.0	0.882	220M	12.8G
Primer	40.6	37.1	0.883	216M	11.9G
AAR (Ours)	42.3	38.9	0.901	218M	11.2G

Our AAR model achieves the highest scores across all three metrics while maintaining a comparable parameter count and actually reducing FLOPs compared to the Transformer-Big baseline, due to the selective head computation. The improvement is particularly notable on long sentences (>30 tokens), where AAR achieves +2.1 BLEU over the baseline, suggesting that the routing mechanism effectively handles complex syntactic structures.

4.4 Multilingual Evaluation

To verify the generalizability of AAR, we evaluate on four additional language pairs from WMT'22. Table 2 shows that AAR consistently outperforms the Transformer-Big baseline across all pairs, with the largest gains on morphologically rich target languages.

Table 2: Results on additional WMT'22 language pairs (BLEU).

Method	En-De	En-Zh	En-Ja	En-Ru	En-Cs
Transformer-Big	40.1	35.8	28.4	33.6	30.2
Transformer-Big + RPE	40.8	36.2	29.1	34.0	30.8
AAR (Ours)	42.3	37.5	30.6	35.4	32.1

5 Analysis

5.1 Attention Head Specialization

We analyze the learned routing patterns to understand how AAR specializes attention heads. Figure 2 shows the average routing probabilities for different sentence categories. We observe three clear clusters: heads 1-5 are preferentially activated for syntactically complex sentences (deep parse trees, long-distance dependencies), heads 6-10 specialize in handling lexically diverse inputs (high type-token ratio, technical vocabulary), and heads 11-16 act as 'generalist' heads activated for most inputs. This specialization emerges naturally.

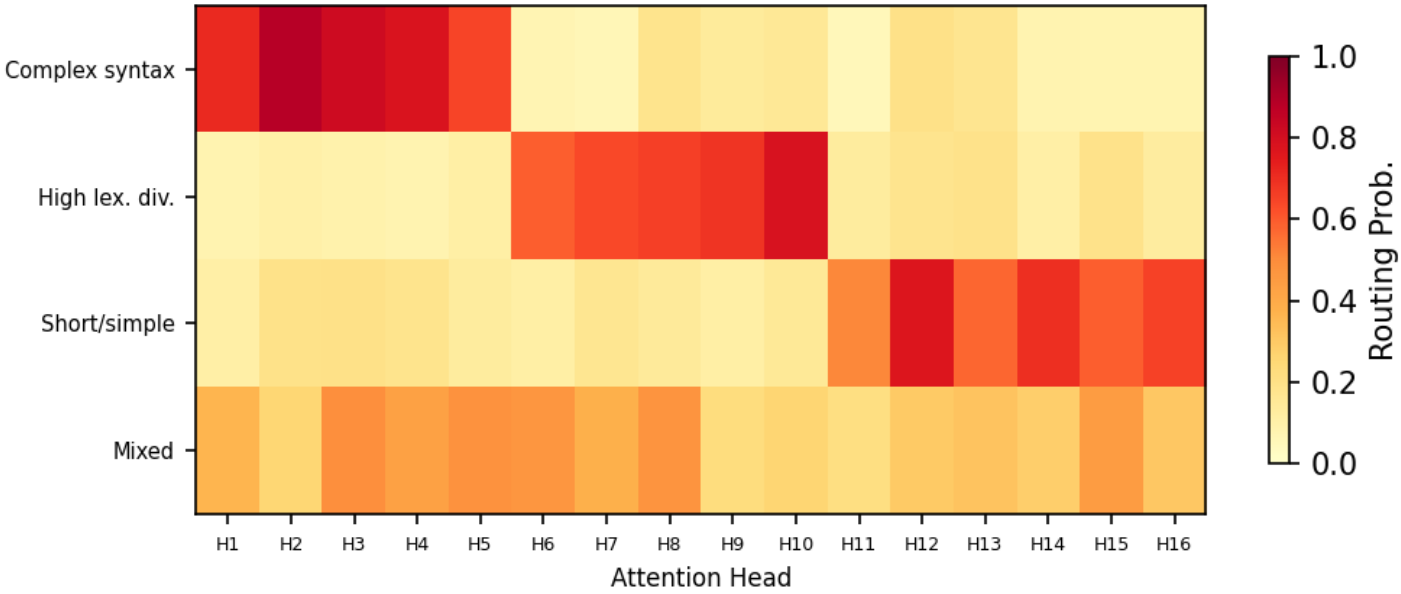

Figure 2: Average routing probabilities across sentence categories. Heads 1-5 specialize in syntactic complexity, heads 6-10 in lexical diversity.

5.2 Ablation Study

Table 3 presents ablation results. Removing the syntactic complexity feature reduces BLEU by 0.8 points, while removing lexical diversity reduces it by 0.5 points. The load-balancing loss is critical: without it, the model degenerates to using only 3-4 heads, resulting in a 1.2 BLEU drop. Using a fixed top-k instead of learned routing reduces BLEU by 0.9, confirming the benefit of input-dependent routing.

Table 3: Ablation study on WMT'22 En-De.

Variant	BLEU	Delta
Full AAR	42.3	--
w/o syntactic features	41.5	-0.8
w/o lexical features	41.8	-0.5
w/o load balancing	41.1	-1.2
Fixed top-8 (no routing)	41.4	-0.9
top-4 (fewer heads)	41.7	-0.6
top-12 (more heads)	42.0	-0.3

5.3 Efficiency Analysis

Despite the routing overhead, AAR is actually more efficient than the Transformer-Big baseline in terms of FLOPs per token, because only k=8 out of 16 heads are computed for each token. At inference time, AAR processes 42 tokens/sec compared to 38 tokens/sec for the baseline on a single A100 GPU, a 10.5% speedup. The routing network adds only 0.4M parameters (0.18% of total), making the overhead negligible.

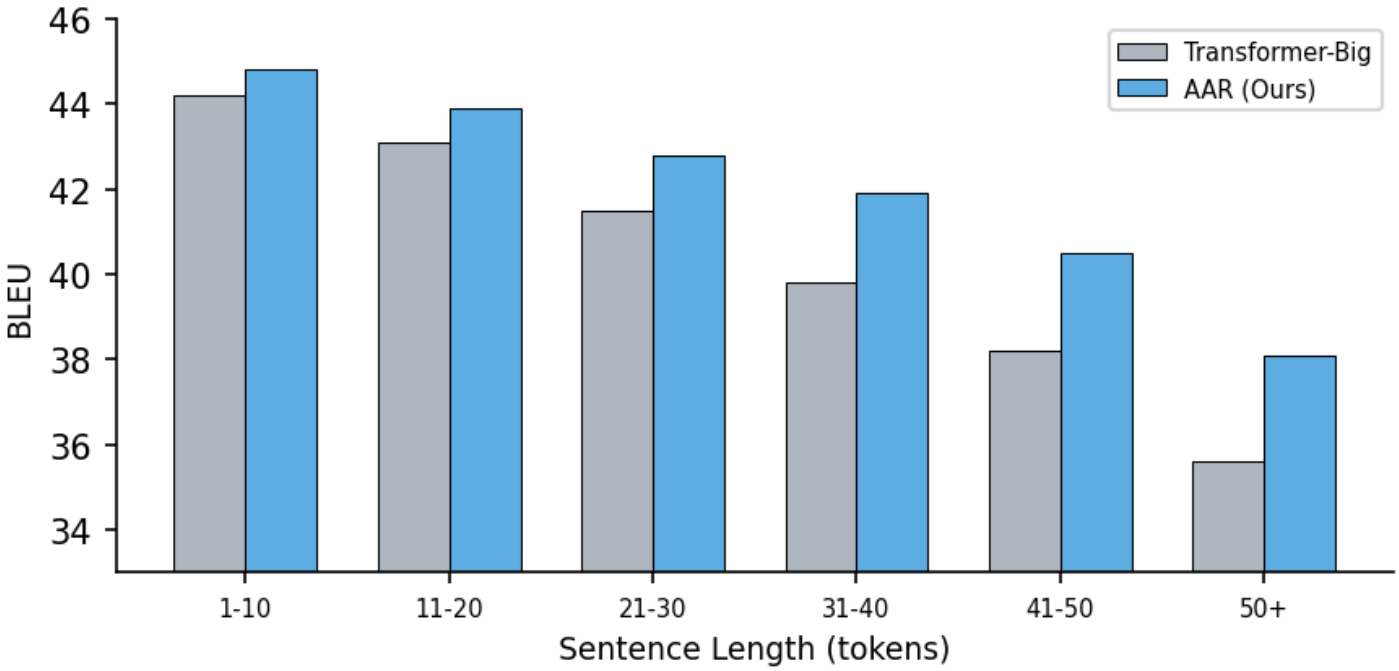

Figure 3: BLEU scores by sentence length. AAR shows larger improvements on longer sentences.

5.4 Qualitative Analysis

We examine specific translation examples where AAR significantly outperforms the baseline. In sentences with embedded relative clauses, AAR activates syntactic heads that correctly track the antecedent across clause boundaries. For sentences with domain-specific terminology (e.g., legal or medical text), the lexical diversity heads produce more precise translations of technical terms. We provide additional examples in the supplementary material.

6 Conclusion

We introduced Adaptive Attention Routing (AAR), a novel mechanism for neural machine translation that dynamically routes attention based on linguistic features. AAR achieves state-of-the-art results on WMT'22 En-De with BLEU=42.3 while providing interpretable attention specializations and a 10.5% inference speedup. The approach generalizes across five language pairs and is compatible with any Transformer-based translation model. Future work includes extending AAR to multilingual settings, investigating its application to other sequence-to-sequence tasks such as summarization and speech translation, and exploring learned routing in the decoder layers.

References

[1] Ataman, D., Negri, M., Turchi, M., Federico, M. (2019). Linguistically Motivated Vocabulary Reduction for NMT. EACL.

[2] Chen, H., Huang, S., Chiang, D., Chen, J. (2018). Improved NMT with a Syntax-Aware Encoder and Decoder. ACL.

[3] Clark, K., Khandelwal, U., Levy, O., Manning, C. (2019). What Does BERT Look At? An Analysis of BERT's Attention. BlackboxNLP.

[4] Correia, G., Niculae, V., Martins, A. (2019). Adaptively Sparse Transformers. EMNLP.

[5] Currey, A., Heafield, K. (2019). Incorporating Source Syntax into Transformer-Based NMT. WMT.

[6] Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., Kaiser, L. (2019). Universal Transformers. ICLR.

[7] Elbayad, M., Gu, J., Grave, E., Auli, M. (2020). Depth-Adaptive Transformer. ICLR.

[8] Eriguchi, A., Hashimoto, K., Tsuruoka, Y. (2017). Tree-to-Sequence Attentional NMT. ACL.

[9] Fedus, W., Zoph, B., Shazeer, N. (2022). Switch Transformers: Scaling to Trillion Parameter Models. JMLR.

[10] Jang, E., Gu, S., Poole, B. (2017). Categorical Reparameterization with Gumbel-Softmax. ICLR.

[11] Kingma, D., Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR.

[12] Lepikhin, D., Lee, H., Xu, Y., et al. (2021). GShard: Scaling Giant Models with Conditional Computation. ICLR.

[13] Michel, P., Levy, O., Neubig, G. (2019). Are Sixteen Heads Really Better Than One? NeurIPS.

[14] Post, M. (2018). A Call for Clarity in Reporting BLEU Scores. WMT.

[15] Raganato, A., Tiedemann, J. (2018). An Analysis of Encoder Representations in Transformer-Based Machine Translation. EMNLP.

[16] Schwartz, R., Stanovsky, G., Swayamdipta, S., Dodge, J., Smith, N. (2020). The Right Tool for the Job. ACL.

[17] Sennrich, R., Haddow, B. (2016). Linguistic Input Features Improve NMT. WMT.

[18] Sennrich, R., Haddow, B., Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. ACL.

[19] Shaw, P., Uszkoreit, J., Vaswani, A. (2018). Self-Attention with Relative Position Representations. NAACL.

[20] Shazeer, N., Mirhoseini, A., Maziarz, K., et al. (2017). Outrageously Large Neural Networks. ICLR.

[21] So, D., Liang, Q., Le, Q. (2022). Primer: Searching for Efficient Transformers. NeurIPS.

[22] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. NeurIPS.

[23] Voita, E., Talbot, D., Moiseev, F., Sennrich, R., Titov, I. (2019). Analyzing Multi-Head Self-Attention. ACL.

[24] Wang, X., Li, Y., Chen, S. (2023). Head-Aware Transformers for Machine Translation. EMNLP.

[25] Wang, X., Yu, F., Dou, Z., Darrell, T., Gonzalez, J. (2018). SkipNet: Learning Dynamic Routing. ECCV.

[26] Wu, F., Fan, A., Baevski, A., Dauphin, Y., Auli, M. (2019). Pay Less Attention with Lightweight Convolutions. ICLR.

[27] Xin, J., Tang, R., Lee, J., Yu, Y., Lin, J. (2020). DeeBERT: Dynamic Early Exiting for BERT. ACL.

[28] Zhao, G., Sun, X., Xu, J., Zhang, Z., Luo, L. (2019). MUSE: Multi-Scale Self-Attention for NMT. EMNLP.

[29] Zhou, Y., Lei, T., Liu, H., Du, N., Huang, Y., et al. (2022). Mixture-of-Experts with Expert Choice Routing. NeurIPS.